

BlockChain Simulator

Candidate name: Ivan Leung

Candidate number: 4022

Centre name: 22245

Qualification code: H446

Year: 2026

Contents

1. Analysis	3
1.1. The Problem	4
1.2. Research	5
1.3. Existing models	9
1.4. Initial Features	12
1.4.1. Project Management Methodology	13
1.4.2. Technology	13
1.4.3. Device Compatibility	13
1.5. Intital Sucess Criteria	14
1.6. Stakeholders	15
1.6.1. Survey	15
1.6.2. Interview	15
2. Iterations	16
2.1. Iteration 1	17
2.1.1. Goal	17
2.1.2. Design	17
2.2. Decomposition	18
2.2.1. Testing	18
2.2.2. Evaluation	18
2.3. Iteration 2	19
2.3.1. Testing	19
2.3.2. Evaluation	19
2.4. Iteration 3	20
2.4.1. Testing	20
2.4.2. Evaluation	20
2.5. Iteration 4	21
2.5.1. Testing	21
2.5.2. Evaluation	21
2.6. Iteration 5	22
2.6.1. Testing	22
2.6.2. Evaluation	22
2.7. Algorithms	23
2.8. Test Data	24
2.9. Data Validation	25
3. Evaluation	26

1. Analysis

1.1. The Problem

Blockchain is a rapidly growing technology that underpins cryptocurrencies, NFTs, and secure digital record-keeping. Blockchain and Cryptography are among the most in-demand STEM skills in the post-web economy. According to a new report from POST, UK Parliament, nearly half of engineering and tech businesses are struggling to recruit skilled talent. STEM skills shortages are costing the UK economy £1.5 billion per year ^[1]. Despite its increasing importance in finance, computing, and data security, understanding how blockchain works is conceptually challenging for most students. Key concepts such as distributed ledgers, mining, proof-of-work, hashing, and peer-to-peer networks are often abstract and difficult to visualise. This creates a barrier to learning, especially when students encounter blockchain only through media or cryptocurrency hype, without practical experience. Consequently, there is a need for an educational tool that allows students to interact with and visualise blockchain mechanics in a safe, simplified, and engaging way.

¹source: medium (<https://britblockchain.medium.com/uk-faces-stem-talent-shortage-as-demand-for-blockchain-skills-surges-says-parliamentary-report-ca626c0f5928>)

1.2. Research

Blockchain is a distributed ledger technology that records transactions across a network of nodes. Each block is a container for multiple transactions, linked to the previous block via a cryptographic hash. New blocks are added through mining, a process where participants solve a computational puzzle (Proof-of-Work). Miners are incentivized with rewards, such as cryptocurrency and transaction fees, for successfully adding a valid block to the chain.

Research in Blockchain Technology:

I gained knowledge on the blockchain technologies after watching this YouTube video ^[2] by 3Blue1Brown which explains how blockchain technologies work behind the scene and different algorithms to power it. I also confirmed my knowledge with the winner of award of distinction in the Hong Kong BlockChain Olympiad - Lester Chong by asking him questions to clearly understand the concepts behind blockchain technologies.

Here are my summary to some main concepts in Blockchain technology:

1. Transactions

A transaction represents a transfer of value or information between users. Each transaction typically includes:

- Sender and receiver addresses
- Amount or data being transferred
- Digital signature to verify authenticity

Transactions are first broadcast to the network and stored in the mempool, a collection of unconfirmed transactions awaiting inclusion in a block.

2. Transaction Validity

Transactions must satisfy several criteria before being considered valid. Miners only include valid transactions in blocks.

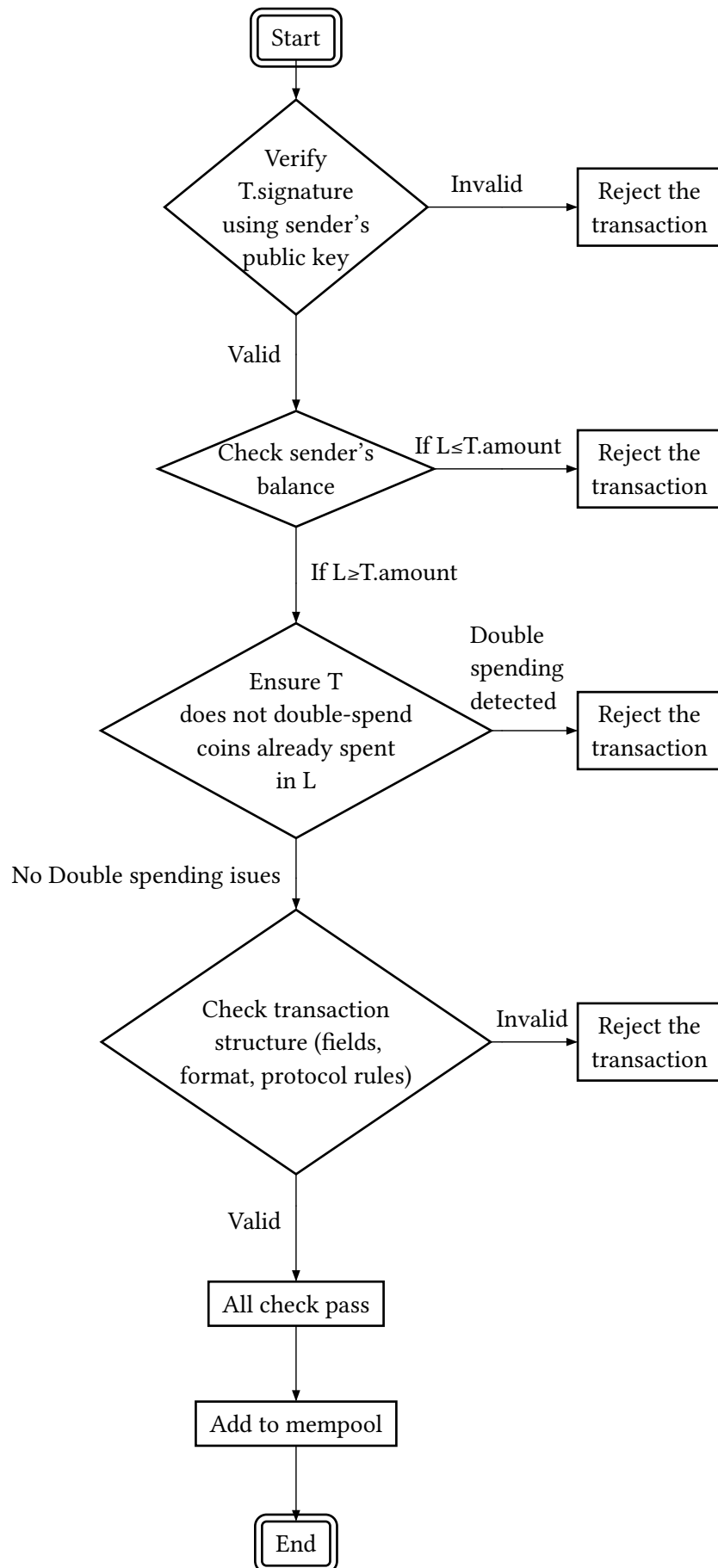
Algorithm: Validate Transaction

Input: Transaction T, Blockchain ledger L

Output: Valid or Invalid

Algorithm flowchart - see next page

²<https://youtu.be/bBC-nXj3Ng4?si=7UgRxuPQdJOzkzG4>



3. Block and block structure

A block is a container for transactions and metadata. Each block contains:

1. Multiple transactions from the mempool
2. Previous block hash — linking the block to its predecessor
3. Timestamp — recording creation time
4. Nonce — Number used Once; adjusted during mining to solve the Proof-of-Work puzzle
5. Block hash — a cryptographic hash of all block contents

4. Hashing

Blocks are uniquely identified using a cryptographic hash function (e.g., SHA-256).

Algorithm: Compute Block Hash Input: Block B (transactions, previous hash, timestamp, nonce)

Output: Block hash H

1. Concatenate block transactions, previous hash, timestamp, and nonce
2. Apply SHA-256 hash function
3. Output fixed-length hash

Properties of cryptographic hashes:

- Deterministic: Same input always produces the same hash
- Irreversible: Impossible to deduce the original input from the hash
- Sensitive: Any change in the input drastically changes the output (avalanche effect)

5. Mining and Proof of Work

Mining involves finding a nonce so that the block hash meets a difficulty target, typically requiring a certain number of leading zeros in binary. One of the mining algorithm is the proof of work mining.

Algorithm: Proof of Work Mining

Input: Block B, Difficulty D

Output: Valid nonce, block hash

```
// Pseudocode algorithm
1. Set nonce = 0
2. Repeat:
  a. Compute hash H = SHA-256(B.transactions + B.previousHash + B.timestamp + nonce)
  b. Convert H to binary
  c. Count leading zeros LZ in binary H
  d. If LZ ≥ D:
    - Stop: valid nonce found
  e. Else:
    - Increment nonce by 1
3. Return valid nonce and hash H
```

When a valid nonce (an arbitrary number that can be used just once in a cryptographic communication) is found, the block is broadcast to the network for inclusion in the blockchain. There are also other mining algorithms such as the proof of stake algorithm.

6. Peer to Peer network

Blockchain relies on a decentralized peer-to-peer (P2P) network. Each node maintains a copy of the blockchain and independently verifies transactions and blocks.

Transaction propagation in P2P network:

Input: Transaction T

Output: Broadcast to network

1. User signs and sends T to a connected node
2. Node verifies T:
 - If valid, add to local mempool
 - If invalid, discard
3. Node forwards T to all connected peers
4. Iterate steps 2-3 for each peer

Block propagation in p2p network:

Input: Mined Block B

Output: Network consensus

1. Miner broadcasts B to connected peers
2. Each peer verifies:
 - a. Block hash meets difficulty
 - b. Transactions are valid
 - c. Previous hash matches last block in local chain
3. If valid, peer appends B to local blockchain
4. Peer forwards B to all its connected peers until it reaches everyone in the network

7. Block linking and immutability

- Each block contains the hash of the previous block, forming a cryptographically linked chain.
- Altering a block invalidates all subsequent blocks, making tampering computationally infeasible.
- Only blocks meeting the Proof-of-Work and containing valid transactions are accepted by the network.

Conceptual Flowchart I have also created the flowchart below to

1.3. Existing models

I found this blockchain simulator Command Line Interface (CLI) on GitHub (<https://github.com/0xs34n/blockchain>) by Sean. I have forked the repository and ran it on my local machine using `node.js`. The simulator offers a basic understanding of blockchain technologies with features like possessing blockchains and connecting to peers in different networks. However, it doesn't contain features such as transactions of blockchains or the process of mining blockchains.

```
ivanextrastuff@Acer-Aspire:~/code/b_simulator/blockchain$ npm start

> blockchain-cli@2.0.0 start /home/ivanextrastuff/code/b_simulator/blockchain
> node ./index.js

Welcome to Blockchain CLI!

Commands:

  help [command...]  Provides help for a given command.
  exit               Exits application.

blockchain → help

Commands:

  help [command...]  Provides help for a given command.
  exit               Exits application.
  connect <host> <port> Connect to a new peer. Eg: connect localhost 2727
  discover            Discover new peers from your connected peers.
  blockchain          See the current state of the blockchain.
  peers              Get the list of connected peers.
  mine <data>         Mine a new block. Eg: mine hello!
  open <port>         Open port to accept incoming connections. Eg: open 2727

blockchain → mine IvanBlock1
blockchain → mine IvanBlock2
blockchain → blockchain
Blockchain {
  blockchain:
    [ Block {
      index: 0,
      previousHash: '0',
      timestamp: 1508270000000,
      data: 'Welcome to Blockchain Demo 2.0!',
      hash: '000dc75a315c77a1f9c98fb6247d03dd18ac52632d7dc6a9920261d8109b37cf',
      nonce: 604 },
      Block {
        index: 1,
        previousHash: '000dc75a315c77a1f9c98fb6247d03dd18ac52632d7dc6a9920261d8109b37cf',
        timestamp: 1757326219903,
        data: 'IvanBlock1',
        hash: '000487dbd92876c509ca612969413c9398cd7a0a936ef1a716ef3d634c3d93c5',
        nonce: 5482 },
      Block {
        index: 2,
        previousHash: '000487dbd92876c509ca612969413c9398cd7a0a936ef1a716ef3d634c3d93c5',
        timestamp: 1757326223309,
        data: 'IvanBlock2',
        hash: '000c5f9316b3c17b07ce6f9fe440ef29036529d0268a55af9457e8a43e413b09',
        nonce: 8579 } ],
  difficulty: 3 }
blockchain →
```

```

ivanextrastuff@Acer-Aspire:~/code/b_simulator/blockchain$ npm run start
> blockchain-cli@2.0.0 start
> node ./index.js

Welcome to Blockchain CLI!

Commands:

  help [command...] Provides help for a given command.
  exit               Exits application.

blockchain → help

Commands:

  help [command...] Provides help for a given command.
  exit               Exits application.
  connect <host> <port> Connect to a new peer. Eg: connect localhost 2727
  discover            Discover new peers from your connected peers.
  blockchain          See the current state of the blockchain.
  peers              Get the list of connected peers.
  mine <data>         Mine a new block. Eg: mine hello!
  open <port>         Open port to accept incoming connections. Eg:
                     open 2727

blockchain → open 2024
Listening to peers on 2024
blockchain → peers
null

blockchain → discover
blockchain →

/home/ivanextrastuff/code/b_simulator/blockchain/P2p.js:39
  throw err;
  ^
Error: Peer did not return any candidates
    at /home/ivanextrastuff/code/b_simulator/blockchain/node_modules/peer-exchange/li
b/swarm.js:258:21
    at PXP.<anonymous> (/home/ivanextrastuff/code/b_simulator/blockchain/node_modules
/pxp/lib/peer.js:439:9)
    at Object.onceWrapper (events.js:421:26)
    at PXP.emit (events.js:314:20)
    at PXP.onMessage (/home/ivanextrastuff/code/b_simulator/blockchain/node_modules
/pxp/lib/pxp.js:66:21)
    at Pumpify.emit (events.js:314:20)
    at addChunk (/home/ivanextrastuff/code/b_simulator/blockchain/node_modules/readab
le-stream/lib/_stream_readable.js:201:12)
    at readableAddChunk (/home/ivanextrastuff/code/b_simulator/blockchain/node_module
s/readable-stream/lib/_stream_readable.js:278:11)
    at Pumpify.Readable.push (/home/ivanextrastuff/code/b_simulator/blockchain/node_m
odules/readable-stream/lib/_stream_readable.js:245:10)
    at Pumpify.Duplexify._forward (/home/ivanextrastuff/code/b_simulator/blockchain/n
ode_modules/duplexify/index.js:170:26)
ivanextrastuff@Acer-Aspire:~/code/b_simulator/blockchain$

ivanextrastuff@Acer-Aspire:~/code/b_simulator/blockchain$ npm run start
> blockchain-cli@2.0.0 start
> node ./index.js

Welcome to Blockchain CLI!

Commands:

  help [command...] Provides help for a given command.
  exit               Exits application.

blockchain → mine IvanBlock
blockchain → blockchain
Blockchain {
  blockchain: [
    Block {
      index: 0,
      previousHash: '0',
      timestamp: 1508270000000,
      data: 'Welcome to Blockchain Demo 2.0!',
      hash: '000dc75a315c77a1f9c98fb6247d03dd18ac52632d7dc6a9920261d8109b37cf',
      nonce: 604
    },
    Block {
      index: 1,
      previousHash: '000dc75a315c77a1f9c98fb6247d03dd18ac52632d7dc6a9920261d8109b37cf',
      timestamp: 1758034508869,
      data: 'IvanBlock',
      hash: '00053d4d9787be5aff2d27dc025ffae15dde1f769a4bba00d7ca64c11b25f93',
      nonce: 1835
    }
  ],
  difficulty: 3
}
blockchain → connect localhost 2024
blockchain → peers
localhost

blockchain →

```

In this image, Sean's Blockchain simulator has shown the connection between different ports in local host. However, there is not any features that allow different ports to interact, like trading blocks. It also doesn't allow user to see other's user blocks.

1.4. Initial Features

In this BlockChain Simulator project, I will build a simplified model of BlockChain and mainly focusing on visualisation of different technologies, so that learners would be able to easily understand them.

The final project will be running on a browser and has a Graphical User Interface (GUI). The project will be developed in HTML, CSS, and JavaScript with GPU.js to gain control over the user's Graphical Processing Unit (GPU). This allows the code to be rendered faster and also better simulate how blockchains are being mined as this is usually being done by a crypto mining rig - a customised personal computer that uses GPUs to solve cryptographic equations and verify transactions on a blockchain.

Features

1. Core Blockchain Mechanics

- Block structure → index, timestamp, list of transactions, previous hash, nonce, and current hash, implemented by Object Oriented Programming OOP
- Hashing algorithm → simple SHA-256 function
- Mining algorithm → find a nonce such that block hash starts with N zeros (difficulty)
- Transaction system → sender, receiver, amount stored in a mempool before being mined, also implemented by OOP
- Chain validation → check each block's previous_hash matches the last block's hash

2. Network Simulation (Peer-to-Peer)

- Each node has its own blockchain copy
- Nodes are connected in an undirected graph
- When a transaction or block is created, it is propagated to all nodes via Breadth First Search (BFS) or Depth First Search (DFS) traversal
- New nodes can join by connecting to at least one existing node

3. Visualisation

- Blockchain view → show blocks as boxes linked in a chain.
- Each block displays: index, hash, previous hash, transactions.
- Mempool view → list of pending transactions waiting to be mined.

4. Abstraction

- Ignoring Double Spending Issues during Chain Validation
- No digital signatures (trust all transactions)
- No forks (always extend the longest chain)
- Assume honest nodes (no attackers)

Controls (buttons):

- “Add Transaction” → enter sender, receiver, amount
- “Mine Block” → mines transactions into a block
- “Propagate Transaction” → spreads it across nodes
- Network view → graph of nodes (circles), with edges showing connections

1.4.1. Project Management Methodology

The solution will be developed in an agile way. This means that it will be developed in different iterations. The analysis of requirements has been done in this section (Section 1). In each iterations I will

- Design the prototype for that iteration
- Build the prototype
- Asking for User Feedback
- Refine prototype

Each feature of the solution might be improved throughout the iterations.

1.4.2. Technology

The simulator will run purely on client side code to reduce server costs and workload, improve scalability, and create a more interactive and responsive user experience

Frontend: HTML + CSS + JavaScript

Graph visualisation: canvas

1.4.3. Device Compatibility

The blockchain simulator is designed to run entirely in the browser and therefore requires JavaScript support to function. It is optimised for modern desktop and laptop environments using Chromium-based browsers (Google Chrome, Microsoft Edge, Opera), as well as Mozilla Firefox and Safari. Mobile browsers may support basic interaction, but performance and visualisation features are best experienced on computer systems.

1.5. Intital Sucess Criteria

These are the initial Success Criteria (SC) which is what I am aiming for while developing a MVP (Minimum Viable Product). As I am developing in an agile methodology, the Success Critea might be ammended or new Success Criteria might be added in the process of iterations.

Success Criteria

SC	Target	Testing
1.1	GUI	$\frac{\sqrt{2}}{12}a^3$
1.2		

1.6. Stakeholders

Primary Stakeholders:

Students: They are the main users of the simulator

1.6.1. Survey

1.6.2. Interview

2. Iterations

2.1. Iteration 1

In iteration 1, I will be doing proof of concept for Breadth First Search (BFS) and Depth First Search (DFS) so that I can later use them for visualising the broadcast of blockchain within the network.

Breadth First Search is an algorithm to traverse an undirected graph.

Depth First Search is another algorithm to traverse an undirected graph.

2.1.1. Goal

2.1.2. Design

The figure below shows the user interface design of different sections of the platform, including the Introduction Page, Main Page, Users Page, Chains Page, Mining Page, Transactions Page, and the Settings Page.

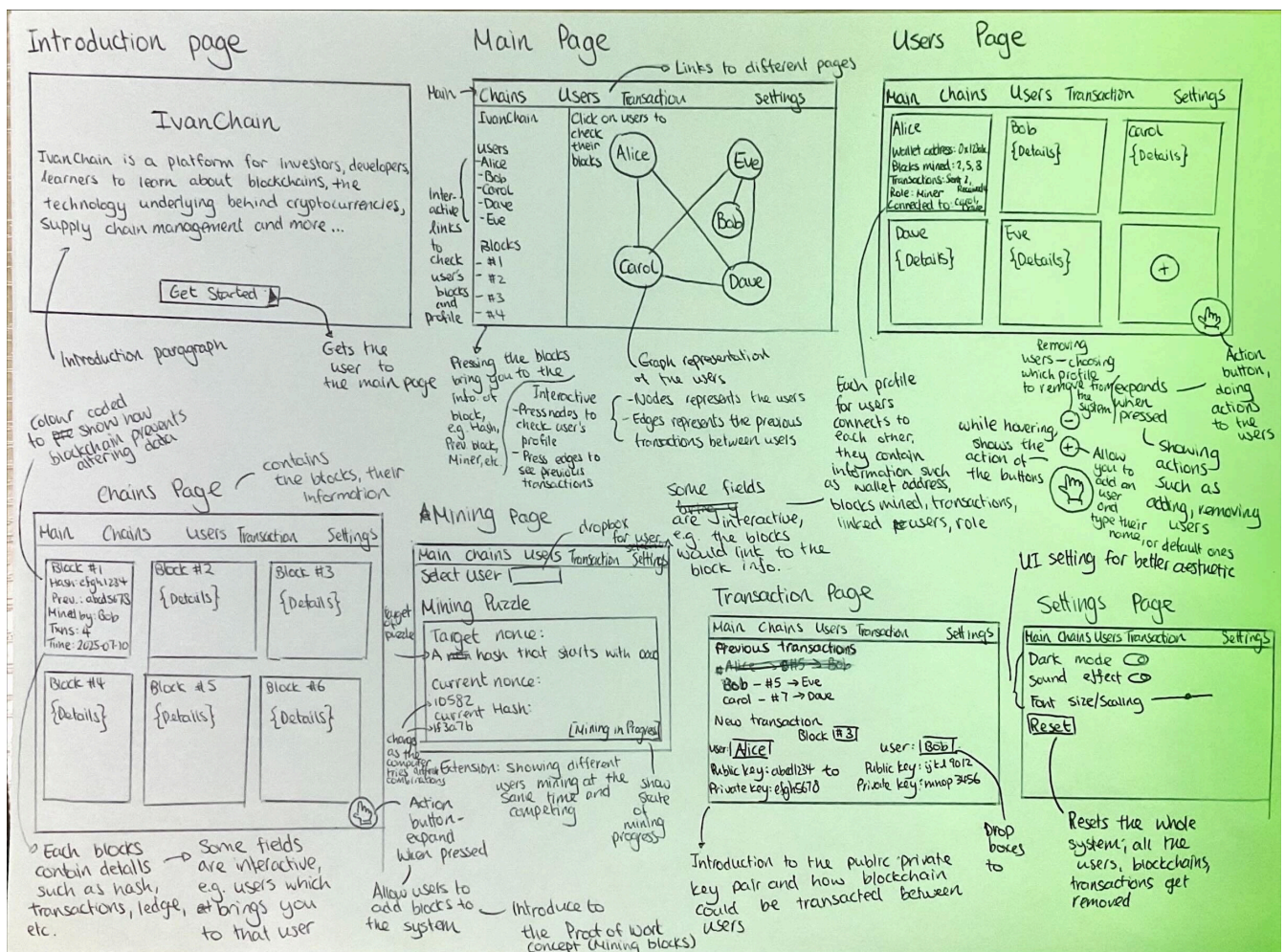


Figure 1: User interface of the blockchain simulator.

2.2. Decomposition

TODO:Justify for decomposition

2.2.1. Testing

2.2.2. Evaluation

2.3. Iteration 2

2.3.1. Testing

2.3.2. Evaluation

2.4. Iteration 3

2.4.1. Testing

2.4.2. Evaluation

2.5. Iteration 4

2.5.1. Testing

2.5.2. Evaluation

2.6. Iteration 5

2.6.1. Testing

2.6.2. Evaluation

2.7. Algorithms

2.8. Test Data

2.9. Data Validation

3. Evaluation